Documentation technique – Module de gestion des informations

Projet PPE – Amicale du Val de Somme

1. Introduction

Dans le cadre du projet PPE réalisé pour l'Amicale du Val de Somme, un module de gestion de contenu a été développé afin de permettre aux administrateurs du site de créer, modifier et supprimer des actualités ou des annonces. Ce module est accessible via l'onglet « **Mes Fonctions** » du back-office, sous l'intitulé « **Gérer les informations** ».

Le présent document décrit l'architecture technique, la structure de la base de données, les interfaces d'administration et le mécanisme d'affichage public de ce module.

2. Accès et interface de gestion

2.1. Page de liste des informations

L'URL d'accès (réservée aux utilisateurs authentifiés disposant des droits adéquats) affiche un tableau listant l'intégralité des enregistrements présents dans la base de données. Les colonnes affichées sont les suivantes :

Colonne	Description
ID	Identifiant unique de l'enregistrement
Titre	Titre de l'information
Type	Visibilité (`Publique` ou `Privée`)
Auteur	Nom de l'utilisateur ayant créé l'entrée
Création	Date et heure de création
Dernière MAJ	Date et heure de la dernière modification

Fichiers	Nombre de documents PDF attachés (ou « Aucun »)
Actions	Icône de suppression (X)

Un bouton **« Ajouter une information »** permet d'accéder au formulaire de création. La suppression d'un élément est immédiate et déclenche également la suppression des éventuels fichiers PDF associés sur le serveur.

3. Structure de la base de données

Les informations sont stockées dans une table nommée `informations`. Sa structure SQL est détaillée ci-dessous.

#	Nom	Type	Attributs	Null	Valeur par défaut	Commentaire
1	`id`	`int`	PRIMARY KEY, AUTO_INCREMENT	Non	–	Identifiant unique
2	`titre`	`varchar(255)`	–	Non	–	Titre de l'information
3	`contenu`	`longtext`	–	Non	–	Contenu textuel riche (HTML)
4	`type`	`enum('Publique','Privé')`	–	Non	`'Publique'`	Niveau de visibilité
5	`auteur`	`varchar(255)`	–	Oui	`NULL`	Nom de l'auteur

6	`date_creation`	`datetime`	DEFAULT `CURRENT_TIMESTAMP`	Non	`CURRENT_TIMESTAMP`	Horodatage à la création
7	`date_modif`	`datetime`	ON UPDATE `CURRENT_TIMESTAMP`	Oui	`NULL`	Horodatage de dernière modification

Remarques techniques :

- Le champ `type` est implémenté sous forme d'ENUM pour garantir l'intégrité des valeurs.
- `date_creation` est automatiquement renseigné à l'insertion grâce à la valeur par défaut `CURRENT_TIMESTAMP`.
- `date_modif` est automatiquement mis à jour lors de chaque modification grâce à la clause `ON UPDATE CURRENT_TIMESTAMP`.

4. Formulaire d'ajout / modification d'une information

4.1. Champs du formulaire

Le formulaire, accessible après clic sur « **Ajouter une information** », contient les éléments suivants :

Champ	Type	Obligatoire	Description
Titre	`input[type=text]`	Oui	Titre affiché en tête de l'information.
Type	`select`	Oui	Choix entre `Publique` (visible par tous) et `Privée` (membres).
Contenu	`textarea` (éditeur WYSIWYG)	Oui	Description détaillée, supportant le formatage HTML.

Documents PDF	`input[type=file]` multiple	Non	Permet d'associer un ou plusieurs fichiers PDF à l'information.

4.2. Gestion des pièces jointes

Les fichiers PDF téléversés sont stockés dans un répertoire dédié sur le serveur (ex. `/uploads/informations/`). Un tableau récapitulatif affiche les fichiers déjà attachés avec les colonnes **Voir** (lien de visualisation), **Nom du fichier**, **Date d'ajout** et **Action** (suppression du fichier seul).

L'enregistrement du formulaire déclenche :

- L'insertion ou la mise à jour de l'enregistrement dans la table `informations`.
- Le déplacement des fichiers temporaires vers le répertoire définitif et l'enregistrement de leurs métadonnées dans une table de liaison (non détaillée ici).

5. Affichage public

Lorsqu'une information de type **Publique** est enregistrée, elle devient automatiquement visible sur la page d'accueil du site dans le bloc **« Dernières nouvelles »**. L'affichage inclut :

- Le **titre** de l'information.
- Un badge **« Nouveau »** si l'information a moins de 7 jours (paramétrable).
- La **date de publication** et le **nom de l'auteur** (ex. : *Publié le 10/04/2026 • JEREMY ZON*).

Les informations de type **Privée** ne sont pas listées publiquement ; elles restent accessibles uniquement via l'interface d'administration ou via une page sécurisée réservée aux membres authentifiés.

6. Maintenance et évolutions possibles

- **Sécurité** : Tous les accès à l'interface d'administration sont protégés par une authentification et une vérification des droits utilisateur.

- **Sauvegardes** : Il est recommandé de sauvegarder régulièrement la table `informations` ainsi que le répertoire des fichiers uploadés.
- **Évolutions envisagées** :
  - Ajout d'un champ **date de publication programmée**.
  - Gestion de plusieurs auteurs par information.
  - Catégorisation des informations (actualités, événements, etc.).

7. Implémentation technique – Classe `Information`

Le modèle métier est encapsulé dans une classe `Information` héritant de `Table` (classe abstraite du framework interne). Cette classe centralise la validation, l'accès aux données et la gestion des fichiers PDF associés.

7.1. Configuration et constantes

```
private const CONFIG = [
    'repertoire' => '/data/documentinformation',
    'extensions' => ['pdf'],
    'types'      => ['application/pdf'],
    'maxSize'    => 5 * 1024 * 1024,
    'require'    => false,
    'rename'     => true,
    'sansAccent' => true,
];
public const DIR = RACINE . self::CONFIG['repertoire'];
```

- Les documents PDF sont stockés dans le dossier `/data/documentinformation/` (relatif à la racine du projet).
- Taille maximale : 5 Mo.
- Renommage automatique avec suppression des accents pour éviter les problèmes de système de fichiers.

7.2. Définition des colonnes (constructeur)

Chaque champ de la table `information` est associé à un objet d'entrée (pattern `Input`) assurant la validation :

Colonne	Type d'entrée	Obligatoire	Valeurs acceptées

`titre`	`InputText`	Oui	MaxLength 255
`contenu`	`InputTextarea`	Oui	HTML autorisé
`type`	`InputList`	Oui	`"Publique"`, `"Privée"` (défaut : `"Publique"`)
`auteur`	`InputText`	Non	–

7.3. Méthodes principales

Récupération des données :

- `getAll(): array` – Retourne toutes les informations triées par date de création décroissante. Inclut `contenu` pour pré-remplissage de l'édition.
- `getById(int $id)` – Récupère une information complète par son identifiant.
- `getDocuments(int $idInformation): array` – Liste les documents PDF associés à une information.
- `getType(int $id)` – Renvoie le type (`Publique` / `Privée`) d'une information.

Gestion des documents :

- `ajouterDocument(string $fichier, int $idInformation, string $nomOriginal): int` – Insère un enregistrement dans la table `documentinformation` et retourne l'ID du document.
- `supprimerDocument(int $id): bool` – Supprime le fichier physique et son entrée en base. Retourne `true` si l'opération a réussi.
- `supprimerEnregistrement(int $id): void` – Supprime uniquement l'entrée en base (utilisé en interne).

Suppression complète :

- `delete(int|string $id): void` – Supprime d'abord tous les documents associés (fichiers physiques + entrées) puis l'information elle-même.

7.4. Table associée `documentinformation`

Cette table (non détaillée dans le schéma initial) assure la liaison entre une information et ses fichiers PDF :

Colonne	Type	Description

`id`	int (PK, AI)	Identifiant du document
`fichier`	varchar	Nom unique généré sur le serveur
`nom_original`	varchar	Nom d'origine du fichier téléversé
`idInformation`	int	Clé étrangère vers `information.id`
`date_upload`	datetime	Horodatage de l'ajout (valeur par défaut)

7.5. Utilisation dans les contrôleurs

Les contrôleurs du back-office instancient la classe `Information` et appellent ses méthodes pour :

- Afficher la liste (`Information::getAll()`).
- Charger une fiche en édition (`Information::getById()`).
- Sauvegarder les données du formulaire (méthode `save()` héritée de `Table`).
- Gérer l'upload de fichiers en utilisant la constante `Information::DIR` et la configuration de `self::CONFIG`.
- Supprimer définitivement une information (`$info->delete($id)`).